

End2Race+: Trajectory-Aware Multi-Agent Racing with Safety Improvement

Shagun Patel

1 Introduction

Autonomous racing in multi-agent environments requires the ability to handle complex behaviors such as overtaking, following, and maintaining safe distances.

This project extends the End2Race framework by incorporating trajectory-aware features and a carefully curated dataset to improve both performance and safety.

2 Objective

The objectives of this project are:

- Learn driving behavior using imitation learning
- Enable multi-agent interaction (overtaking and following)
- Improve safety of overtaking maneuvers
- Develop a complete data-driven pipeline from simulation to deployment

3 Software Stack

- ROS2 (Humble)
- Gazebo Simulator
- Python 3.10
- PyTorch

4 System Pipeline

The system is built as a complete pipeline:

1. Simulation in Gazebo
2. Data collection from driving scenarios
3. Dataset organization and filtering
4. Model training (End2Race+)
5. Evaluation and improvement

5 Data Collection

5.1 Overview

Data collection is the most critical component of this project. The model learns entirely from recorded driving behavior, so the quality of the dataset directly determines the quality of the learned policy.

5.2 Simulation Setup

Data is collected in a Gazebo-based racing environment using:

- Two vehicles:
 - Ego vehicle (learning target)
 - Opponent vehicle (reference behavior)
- Predefined racetrack
- LiDAR sensor (360-degree scan)

5.3 Driving Scenarios

Three types of scenarios are recorded:

- **Clear track:** No opponent, pure trajectory following
- **Follow-behind:** Ego vehicle follows a slower opponent
- **Overtaking:** Ego vehicle overtakes the opponent

5.4 Data Logging Process

Data is recorded using the ROS2 node:

```
ros2 run end2race_ros2 dataset_logger
```

Each run generates a CSV file containing time-series data.

5.5 Recorded Features

Each row in the dataset includes:

Ego State

- Position: x, y
- Orientation: yaw
- Speed: ego_speed

Control Commands

- cmd_speed

- cmd_steer

Trajectory Features

- Cross-track error (cte)
- Heading error
- Reference speed (v_ref)

Opponent Information

- Relative position: opp_rel_x, opp_rel_y
- Distance: opp_dist
- Relative position indicator: opp_ahead

Sensor Data

- LiDAR readings: lidar_0 to lidar_359

5.6 Dataset Organization

The dataset is structured as:

```
/data/end2race_gazebo/
clear/
follow/
  good/
  review/
overtake/
  good/
  review/
  aggressive/
```

5.7 Data Filtering and Curation

Raw data is not directly used for training. Instead, it is manually analyzed and categorized:

- **Good:** High-quality behavior
- **Review:** borderline cases
- **Aggressive:** unsafe behavior

This step is crucial because:

- The model learns average behavior from data
- Including unsafe samples leads to unsafe predictions
- Removing unsafe samples improves safety

6 Model: End2Race+

6.1 Input Representation

The model takes:

- 360-dimensional LiDAR input
- Ego speed
- Trajectory features (cte, heading error, v_ref)

6.2 Architecture

- Feature embedding layers
- GRU (temporal learning)
- Fully connected output layer

The GRU allows the model to learn temporal dependencies such as smooth control and consistent behavior.

““latex

7 Training Process

7.1 Training Method

The End2Race+ model was trained using imitation learning. In simple terms, imitation learning means that the model learns by looking at examples of good driving and then tries to copy those actions.

For every timestep in the collected dataset, the model receives the driving situation as input and learns to predict the control command that was used by the expert driver.

The learning problem can be described as:

$$Observation \rightarrow Action \tag{1}$$

In this project:

- The **observation** contains LiDAR, ego speed, and trajectory information.
- The **action** contains steering and speed commands.

The model is not trained by trial and error. Instead, it is trained using recorded data collected from Gazebo simulation.

7.2 Expert Data Used for Training

A hybrid expert setup was used for collecting training data.

- **GBLP** was used for clear-track driving. This helped collect smooth trajectory-following data when no opponent vehicle was involved.
- **End2Race** was used for following and overtaking scenarios. This helped collect multi-agent behavior where the ego vehicle interacts with another vehicle.

This was done because one expert alone was not enough for all scenarios. GBLP is good at following the raceline, while End2Race is better for interaction-based behavior such as following and overtaking.

7.3 Training Data Format

All collected data was saved as CSV files. Each CSV file represents one driving episode.

Each row in the CSV file represents one timestep and contains:

- LiDAR readings from lidar_0 to lidar_359
- Ego vehicle speed
- Cross-track error (CTE)
- Heading error
- Reference speed from raceline
- Expert steering command
- Expert speed command

The most important columns used for training were:

- **Input columns:** LiDAR, ego speed, CTE, heading error, and reference speed
- **Target columns:** cmd_steer and cmd_speed

The goal of training was to make the model predict cmd_steer and cmd_speed as close as possible to the expert commands.

7.4 Training Code Files

The training pipeline was implemented using three main files:

- **dataset_plus.py:** loads CSV files and creates training sequences
- **model_plus.py:** defines the End2Race+ neural network
- **train_plus.py:** trains the model using PyTorch

These files were added separately so that the original End2Race files were not changed.

7.5 Dataset Loading in Code

The file `dataset_plus.py` reads all CSV files from the dataset folder.

The dataset was organized as:

```
~/nav2_ws/data/end2race_gazebo/  
clear/  
follow/  
    good/  
overtake/  
    good/
```

The dataset loader searches these folders, reads each CSV file, checks that the required columns exist, removes invalid values, and then converts the data into training sequences.

This step is important because the neural network cannot directly learn from raw CSV files. The data must first be converted into tensors that PyTorch can use.

7.6 Sequence Creation

The model was trained using sequences of length 50.

This means that instead of giving the model only one timestep, we give it 50 consecutive timesteps.

For example:

`t1, t2, t3, ..., t50`

This is useful because driving depends on what happened recently. Overtaking and following are not single-frame decisions. They happen over time.

By using sequences, the model can learn:

- how steering changes smoothly over time
- how the vehicle approaches another car
- how to follow behind
- how to complete an overtaking maneuver

7.7 Model Input

For each timestep, the model input consists of:

- 360 LiDAR values
- ego speed
- CTE
- heading error
- reference speed

In simple terms, the model is given:

- what the vehicle sees
- how fast it is moving
- where it is compared to the raceline
- how well it is aligned with the raceline
- what speed the raceline suggests

This makes the model trajectory-aware.

7.8 Model Output

The model predicts two values:

- steering command
- speed command

These outputs are later published to the ROS2 command velocity topic during deployment.

7.9 End2Race+ Model Architecture

The model architecture is defined in `model_plus.py`.

The model contains:

- LiDAR preprocessing
- speed embedding layer
- trajectory feature embedding layer
- GRU layer
- output layer

The LiDAR data, speed information, and trajectory features are first processed separately. After that, they are combined and passed into the GRU.

7.10 Why GRU Was Used

A GRU is a neural network layer that has memory.

This is useful because driving depends on previous timesteps. For example, when overtaking, the vehicle needs to remember that it was approaching the opponent, moving around it, and then returning to the raceline.

Without memory, the model would only react to the current timestep. With the GRU, the model can learn smoother and more consistent driving behavior.

7.11 Training Script

The model was trained using `train_plus.py`.

The training command was:

```
cd ~/nav2_ws/src/End2Race

python3 train_plus.py \
--data_root /home/rtec4/nav2_ws/data/end2race_gazebo \
--model_path /home/rtec4/nav2_ws/src/End2Race/pretrained/end2race_plus_safer.pth \
--sequence_length 50 \
--stride 5 \
--batch_size 16 \
--num_epochs 50 \
--include_review \
--include_aggressive
```

7.12 Meaning of Training Arguments

- **data_root**: path to the collected CSV dataset
- **model_path**: location where the trained model is saved
- **sequence_length**: number of timesteps given to the GRU at once
- **stride**: step size used when creating sequences from CSV files
- **batch_size**: number of sequences processed together during training
- **num_epochs**: number of times the model sees the dataset
- **include_review**: includes borderline samples
- **include_aggressive**: includes aggressive samples if needed for robustness

7.13 Loss Function

The model was trained using supervised learning. The predicted steering and speed were compared with the expert steering and speed.

The loss function was:

$$Loss = L_{steer} + 0.05 \cdot L_{speed} \quad (2)$$

Here:

- L_{steer} measures steering prediction error
- L_{speed} measures speed prediction error

Steering was weighted more heavily because steering mistakes are more dangerous in racing. A wrong steering command can cause the vehicle to leave the track or collide. Speed errors are still important, but they are less critical than steering errors.

7.14 Training Process in Code

During training, the following steps happen repeatedly:

1. Load a batch of sequences from the dataset
2. Pass LiDAR, speed, and trajectory features into the model
3. The model predicts steering and speed
4. Compare predicted commands with expert commands
5. Compute loss
6. Use backpropagation to update model weights
7. Save the model if the loss improves

This process continues for all epochs.

7.15 Trained Model Outputs

After training, the final model was saved as:

```
/home/rtec4/nav2_ws/src/End2Race/pretrained/end2race_plus_safer.pth
```

This model was then used in ROS2 through the End2Race+ driver.

7.16 Why This Training Method Was Chosen

This method was chosen because it is stable and practical.

Reinforcement learning requires trial and error, which can be unstable in racing simulations because the vehicle may crash, leave the track, or learn unsafe behavior. Since expert driving behavior was already available from GBLP and End2Race, imitation learning was a better starting point.

This allowed the model to learn useful driving behavior efficiently from expert demonstrations.

7.17 Summary

In summary, the training process converts recorded driving demonstrations into supervised learning data. The End2Race+ model learns to predict steering and speed from LiDAR, speed, and trajectory features. By using sequence learning with a GRU, the model learns smooth and consistent driving behavior for clear driving, following, and overtaking.

8 Testing Methodology and Iterative Model Evaluation

8.1 Why Testing Was Necessary

After training a machine learning model, it is important to verify whether the learned behavior actually works in practice.

A low training loss only indicates that the model successfully learned to imitate the expert demonstrations. However, it does not guarantee that the vehicle will behave safely or effectively when deployed in a racing environment.

For example, a model may successfully learn to overtake another vehicle while still maintaining an unsafe passing distance. Such behavior may not be visible from the training loss alone.

Therefore, a dedicated testing and evaluation pipeline was developed.

8.2 Testing Procedure

The trained model was deployed back into Gazebo and allowed to drive autonomously.

Unlike the training phase, where the model learns from expert demonstrations, the testing phase evaluates the behavior produced by the trained model itself.

The testing pipeline consisted of:

1. Load a trained End2Race+ model.
2. Deploy it as the controller of the ego vehicle.
3. Place an opponent vehicle on the track.
4. Allow the ego vehicle to drive autonomously.
5. Record the resulting behavior using the dataset logger.
6. Analyze the recorded episode using evaluation metrics and reward functions.

The recorded testing episodes were saved as CSV files.

These files contain the behavior generated by the trained model and are referred to as evaluation datasets.

8.3 What is an Evaluation Dataset?

An evaluation dataset is different from a training dataset.

Training datasets contain demonstrations generated by expert controllers such as GBLP or End2Race.

Evaluation datasets contain behavior generated by the trained End2Race+ model itself.

In other words:

- Training data answers the question:
“What should the model learn?”
- Evaluation data answers the question:
“How does the trained model actually behave?”

This distinction is important because the learned behavior may differ from the expert demonstrations.

8.4 Reward-Based Evaluation

Visual inspection alone is often insufficient for evaluating autonomous driving behavior. While it is possible to observe whether a vehicle successfully completes an overtaking maneuver, it is difficult to objectively compare different models using visual observations alone.

To address this limitation, a reward-based evaluation function was introduced. It is important

to note that the reward function was not used as the primary training objective. The End2Race+ model was trained using imitation learning. The reward function was introduced after training as an evaluation tool to quantitatively measure driving performance and safety.

The reward function evaluates multiple aspects of vehicle behavior simultaneously, including:

- Forward progress along the track
- Trajectory tracking quality
- Vehicle orientation relative to the raceline
- Overtaking safety

The reward function is defined as:

$$R = +w_p \cdot \Delta s - w_c \cdot |CTE| - w_h \cdot |heading_error| - w_d \cdot \max(0, d_{safe} - d_{opp}) \quad (3)$$

where:

- Δs represents the progress of the vehicle along the raceline.
- CTE (cross-track error) measures how far the vehicle is from the optimal trajectory.
- $heading_error$ measures the angular difference between the vehicle heading and the raceline heading.
- d_{opp} is the distance between the ego vehicle and the opponent vehicle.
- d_{safe} is the predefined safe overtaking distance.

8.4.1 Progress Reward

The first term rewards forward movement along the raceline.

A vehicle that successfully completes laps and continues progressing around the track receives a higher reward.

Without this term, the vehicle could remain stationary and still avoid penalties.

8.4.2 Cross-Track Error Penalty

The cross-track error penalty discourages the vehicle from moving away from the optimal racing line. A lower cross-track error indicates better trajectory tracking and more stable driving behavior.

8.4.3 Heading Error Penalty

The heading error penalty encourages the vehicle to align its orientation with the direction of the raceline. Even if the vehicle is physically close to the raceline, poor heading alignment may lead to unstable behavior in future timesteps.

This term helps maintain smooth trajectory-following behavior.

8.4.4 Opponent Distance Penalty

The final term is the most important safety component.

$$\max(0, d_{safe} - d_{opp}) \quad (4)$$

This term becomes active only when the ego vehicle gets closer than the predefined safety distance.

If:

$$d_{opp} > d_{safe} \quad (5)$$

then no penalty is applied.

However, if:

$$d_{opp} < d_{safe} \quad (6)$$

the penalty increases as the distance decreases.

This allows the reward function to identify aggressive overtaking maneuvers that may still appear successful from a trajectory-tracking perspective.

8.4.5 Why Reward-Based Evaluation Was Needed

During early experiments, the model successfully completed overtaking maneuvers and achieved low training loss.

However, evaluation revealed that the minimum distance to the opponent during overtaking was often very small.

For example, the initial model achieved overtaking distances of approximately:

$$d_{min} \approx 0.48m \quad (7)$$

Although overtaking was successful, the behavior was considered unsafe.

This issue could not be identified using training loss alone.

The reward-based evaluation function made it possible to quantitatively measure safety and compare different versions of the model.

8.4.6 Implementation

The reward evaluation pipeline was implemented in:

`reward_eval.py`

and executed using:

```
python3 reward_eval.py --csv file.csv
```

The script processes evaluation datasets generated by the trained model and computes:

- Total reward
- Progress score
- Mean cross-track error

- Mean heading error
- Minimum opponent distance
- Number of unsafe close-distance interactions

8.4.7 Role in Dataset Refinement

The reward-based evaluation function played a critical role in improving the final model.

The evaluation process revealed that some overtaking demonstrations were overly aggressive. These demonstrations were then reviewed and categorized into:

- Good
- Review
- Aggressive

Additional safer overtaking demonstrations were collected and added to the dataset.

After retraining with these safer examples, the final model achieved significantly larger overtaking distances while maintaining stable trajectory tracking.

Therefore, the reward function was not only an evaluation tool but also a mechanism for identifying weaknesses in the dataset and guiding dataset refinement.

8.5 Model Descriptions

Three versions of the proposed End2Race+ model were evaluated in this work.

- **Clean:** The baseline model trained using the original expert demonstration dataset (GBLP & End2Race) without any additional data augmentation or filtering.
- **Mixed:** The model retrained using the original demonstration dataset together with additional expert demonstrations collected from new simulation runs. The objective was to increase the diversity of driving behaviors, particularly during following and overtaking scenarios.
- **Safer:** The model retrained using demonstrations selected through reward-function evaluation. Lower-quality demonstrations were discarded, while demonstrations exhibiting safer overtaking behavior and better overall performance were retained for training.

8.6 First Evaluation: Clean Model

The first trained model was evaluated using files such as:

`eval_plus_overtake_01.csv`

This model was trained primarily using the initial dataset.

The objective of this evaluation was to verify:

- whether overtaking was successful
- whether trajectory tracking remained stable

- whether the vehicle maintained a safe distance from the opponent

The evaluation revealed that the model successfully overtook the opponent but often passed too closely.

Typical minimum overtaking distances were approximately:

$$d_{min} \approx 0.48m \quad (8)$$

Although overtaking was successful, this distance was considered unsafe.

This observation became the first indication that the training dataset contained overly aggressive demonstrations.

8.7 Motivation for Dataset Refinement

The evaluation results suggested that the model was learning unsafe overtaking behavior.

The reason was that neural networks learn statistical patterns from the dataset.

If unsafe overtakes are present in the training data, the model may reproduce those unsafe behaviors.

To address this issue, the dataset was manually reviewed.

Recorded overtaking episodes were categorized into:

- Good
- Review
- Aggressive

Unsafe and overly aggressive examples were identified and separated.

Additional safer overtaking demonstrations were collected and added to the dataset.

8.8 Second Evaluation: Mixed Model

After refining the dataset, a second model was trained.

This model used:

- Good demonstrations
- Review demonstrations
- Selected aggressive demonstrations

This model is referred to as the Mixed Model.

The corresponding evaluation files included:

```
eval_plus_mixed_01.csv
eval_plus_mixed_02.csv
eval_plus_mixed_03.csv
```

The purpose of these evaluations was to determine whether additional data diversity improved performance.

Results showed:

- Overtaking remained successful
- Trajectory tracking improved
- Minimum overtaking distance increased

Typical passing distances improved to approximately:

$$d_{min} \approx 0.63m \quad (9)$$

This represented a significant improvement over the Clean Model. However, the vehicle was still overtaking closer than desired.

8.9 Collection of Safer Demonstrations

To further improve safety, additional overtaking demonstrations were recorded. These demonstrations were selected according to:

- Successful overtaking
- Stable trajectory tracking
- Minimum distance greater than approximately 0.7m

Examples included:

```
overtake_12.csv
overtake_13.csv
overtake_14.csv
overtake_15.csv
```

These demonstrations intentionally represented safer overtaking behavior.

The objective was to teach the model to maintain a larger safety margin during passing maneuvers.

8.10 Final Evaluation: Safer Model

After incorporating the safer demonstrations, a final model was trained.

This model is referred to as the Safer Model.

Evaluation files included:

```
final_eval_safer_01.csv
final_eval_safer_02.csv
final_eval_safer_03.csv
```

The objective of this evaluation stage was to determine whether the model learned safer overtaking behavior without sacrificing trajectory tracking performance.

The results showed:

- Successful overtaking
- Stable raceline tracking

- Improved safety margins

Typical overtaking distances increased to:

$$d_{min} \geq 0.70m \tag{10}$$

This represented the safest behavior achieved during the project.

8.11 Key Insight from Evaluation

The evaluation process demonstrated a very important machine learning principle:

The quality of the training dataset directly influences the quality of the learned behavior.

The network did not become safer because the architecture changed.

The network became safer because the demonstrations used for training became safer.

This finding highlights the importance of dataset curation in imitation learning systems.

8.12 Final Model Selection

Based on all evaluation results, the final deployed model was:

`end2race_plus_safer.pth`

This model achieved the best balance between:

- trajectory tracking
- overtaking success
- safety
- smooth control

It therefore became the final version of End2Race+ used in this project.

9 Results

Model	Min Distance	CTE	Heading	Result
Clean	0.48	0.125	0.068	Unsafe
Mixed	0.63	0.106	0.061	Improved
Safer	0.70+	0.088	0.075	Safe

9.1 Key Observations

- Dataset quality strongly affects behavior
- Removing unsafe samples improves safety
- The model learns safe overtaking without reinforcement learning

10 Qualitative Results: Video Demonstrations

Video demonstrations of the overtaking scenarios are available at the following link:

Google Drive Video Folder

The folder contains the following scenarios:

- Case 01: Standard overtaking scenario with stable trajectory tracking
- Case 02: Overtaking with a large speed difference
- Case 03: Overtaking with similar speeds, highlighting more challenging interactions

11 Conclusion

End2Race+ successfully learns multi-agent racing behavior using imitation learning.

Through careful data collection and dataset curation, the system achieves:

- Reliable overtaking
- Stable trajectory tracking
- Improved safety in multi-agent scenarios

12 Future Work

- Multi-track generalization
- Automatic dataset filtering
- Improved reward-based training methods

A Appendix A: Additional Experimental Results

A.1 Frequency Study

A frequency study was conducted to investigate how controller update frequency affects overtaking behavior, trajectory tracking, and overall driving performance.

Controller frequency determines how often the model receives sensor information and computes a new steering and speed command.

The following frequencies were evaluated:

- 5 Hz (one control update every 200 ms)
- 10 Hz (one control update every 100 ms)
- 20 Hz (one control update every 50 ms)

The motivation behind this experiment was to understand whether increasing the controller update rate allows the vehicle to react more quickly to environmental changes and therefore improve overtaking performance and safety.

For each experiment:

- Vehicle 1 was controlled by the model being evaluated.
- Vehicle 2 was controlled using the original End2Race model.
- The same racetrack and simulation environment were used.
- A complete overtaking maneuver was recorded.
- The resulting CSV file was evaluated using the reward evaluation pipeline.

A.2 Metric Definitions

The following metrics are reported throughout the frequency study.

- **Reward**

Overall performance score produced by the reward evaluation function. Higher values indicate better overall driving performance.

- **Minimum Distance**

The smallest distance observed between the ego vehicle and the opponent during overtaking. Larger values generally indicate safer overtaking behavior.

- **CTE (Cross-Track Error)**

Average distance between the vehicle and the global raceline. Smaller values indicate better trajectory tracking performance.

- **Heading Error**

Average angular difference between the vehicle heading and the raceline heading. Smaller values indicate better alignment with the reference trajectory.

- **Too Close Count**

Number of timesteps where the vehicle came closer than the predefined safety threshold.

- **Collision-like Count**

Number of timesteps where the distance became critically small and was classified as potentially unsafe.

A.3 End2Race Frequency Results

Table 1 summarizes the results obtained using the original End2Race model.

Table 1: End2Race Frequency Evaluation						
Frequency	Reward	Min Dist	CTE	Heading	Too Close	Collision-like
5 Hz	84.10	0.531	0.094	0.081	14	0
10 Hz	-4.62	0.425	0.092	0.086	14	4
20 Hz	106.35	0.827	0.096	0.070	0	0

The original End2Race model exhibited noticeably different behavior across frequencies.

The 10 Hz experiment produced the lowest reward and the smallest overtaking distance, indicating more aggressive behavior and several collision-like interactions.

The 20 Hz configuration achieved the highest reward, the largest overtaking distance, and zero collision-like events. This suggests that the original End2Race model benefits from more frequent control updates.

A.4 End2Race+ Frequency Results

Table 2 summarizes the results obtained using the proposed End2Race+ model.

Table 2: End2Race+ Frequency Evaluation						
Frequency	Reward	Min Dist	CTE	Heading	Too Close	Collision-like
5 Hz	75.65	1.074	0.123	0.079	0	0
10 Hz	54.88	0.529	0.119	0.062	15	0
20 Hz	91.50	0.599	0.112	0.085	16	0

The End2Race+ model demonstrated a different trend.

The largest overtaking distance was observed at 5 Hz, where the vehicle maintained a minimum distance of approximately 1.07 meters from the opponent vehicle.

All three frequency configurations successfully completed overtaking maneuvers without collision-like events.

Compared to the original End2Race model, End2Race+ generally maintained larger overtaking clearances, suggesting safer overtaking behavior.

A.5 Frequency Study Discussion

The frequency study indicates that controller update frequency influences overtaking behavior, trajectory tracking, and safety margins.

The original End2Race model appears to benefit from higher control frequencies, while End2Race+ demonstrates larger overtaking clearances at lower frequencies.

Since only a single overtaking run was performed for each frequency configuration, these results should be considered preliminary. Additional trials are required before drawing statistically significant conclusions regarding the optimal controller frequency.

A.6 Inference Latency Evaluation

In addition to overtaking performance, inference latency was measured to evaluate the computational cost of the proposed End2Race+ architecture.

Inference latency measures the time required for the model to process sensor information and generate a steering and speed command.

Lower latency indicates faster decision making and improved responsiveness.

Both models were evaluated using the same hardware and software configuration.

Table 3: Inference Latency Comparison

Model	Mean (ms)	Median (ms)	Min (ms)	Max (ms)
End2Race	20.44	19.16	2.11	45.94
End2Race+	21.28	21.05	7.77	34.49

The results show that End2Race+ introduces only a small increase in computational cost.

The average latency increased from approximately 20.44 ms to 21.28 ms, corresponding to less than 1 ms of additional processing time.

Given the 100 ms control period used during deployment at 10 Hz, both models comfortably satisfy real-time requirements.

These results indicate that the additional trajectory-aware processing components introduce negligible runtime overhead while enabling improved overtaking behavior.

B Appendix B: Additional Experimental Results

To further investigate the influence of trajectory information on the driving policy, additional experiments were conducted using four different models:

- **End2Race**: Original imitation learning model proposed in the baseline work.
- **End2Race-OT**: Proposed model trained using optimized-trajectory demonstrations together with interaction (following and overtaking) demonstrations.
- **End2Race-ClearOnly**: Model trained exclusively using optimized trajectory (clear-track) demonstrations.
- **End2Race-TG**: Proposed trajectory-guided architecture employing independent feature encoding branches before feature fusion.

Two evaluation settings were considered:

1. Single-vehicle navigation, used to evaluate trajectory tracking performance.
2. Two-vehicle racing, used to evaluate overtaking capability and interaction behavior.

Evaluation Metrics

The following metrics are used to compare the performance of all evaluated models.

- **Lap Time (s):** The total time required for the ego vehicle to complete one full lap. Lower lap times indicate faster overall racing performance.
- **Mean Speed (m/s):** The average speed of the ego vehicle while it is moving. Higher values indicate that the vehicle is able to maintain greater racing speeds.
- **Mean Cross-Track Error (CTE) (m):** The average perpendicular distance between the ego vehicle and the reference (optimized) racing trajectory. Lower values indicate better trajectory-following accuracy.
- **Mean Heading Error (rad):** The average difference between the vehicle heading and the heading of the reference trajectory. Lower values indicate better alignment with the desired racing direction.
- **Reward:** The cumulative reward computed using the proposed evaluation function, which considers trajectory tracking, progress, safety, and smooth control. Higher rewards indicate better overall driving performance.
- **Progress:** The total distance travelled along the reference trajectory during an evaluation episode. Larger values indicate greater forward progress.
- **Minimum Opponent Distance (m):** The smallest distance maintained between the ego vehicle and the opponent vehicle throughout the experiment. Larger values generally indicate safer interaction.
- **Too-Close Count:** The number of timesteps during which the distance between the ego vehicle and the opponent falls below the predefined safety threshold (0.7 m). Lower values indicate safer driving behaviour.
- **Collision-like Count:** The number of timesteps during which the opponent distance falls below the collision threshold (0.35 m). A value of zero indicates that no collision-like situations occurred.
- **Smoothness Penalty:** A measure of the cumulative steering variation during driving. Lower values indicate smoother steering behaviour and more stable vehicle control.
- **CPU Utilization (%):** The average processor utilization measured while the model was running in real time. Lower utilization indicates higher computational efficiency.
- **Inference Latency (ms):** The average time required by the model to generate a steering and speed command for one control cycle. Lower latency is desirable for real-time autonomous racing.

The additional experiments provide several important observations.

The End2Race-OT model achieved the lowest average cross-track error among the interaction models while maintaining zero collision-like events, indicating improved trajectory tracking without sacrificing overtaking performance. Furthermore, the steering smoothness penalty was substantially

Table 4: Single-vehicle evaluation results.

Model	Lap Time (s)	Mean Speed (m/s)	Mean CTE (m)	Heading Error (rad)
End2Race	61.15	3.399	0.0871	0.0731
End2Race-OT	57.10	3.398	0.0917	0.0638
ClearOnly	57.00	3.388	0.0909	0.0399
Trajectory-Guided	56.55	3.394	0.0905	0.0649

Table 5: Two-vehicle interaction evaluation results (mean values over three runs).

Model	Mean CTE	Heading Error	Min Distance (m)	Too Close	Smoothness
End2Race	0.0953	0.0640	0.757	0	6.99
End2Race-OT	0.0876	0.0620	0.745	0	2.95
ClearOnly	0.0888	0.0365	0.442	10.67	5.73
Trajectory-Guided	0.0953	0.0596	0.642	5.67	3.59

reduced compared to the original End2Race model, suggesting more stable control actions.

The ClearOnly model demonstrated good trajectory tracking performance in the single-vehicle setting; however, its performance degraded considerably during two-vehicle interaction. Since the model was trained exclusively on clear-track demonstrations, it lacked sufficient experience in handling dynamic opponents, resulting in a larger number of close interactions.

The proposed Trajectory-Guided (TG) architecture achieved the shortest lap time among the evaluated models while maintaining competitive trajectory-tracking performance and smoother steering behavior than the baseline. Although the current implementation still exhibits room for improvement in interaction scenarios, these preliminary results indicate that explicitly separating LiDAR, trajectory, and vehicle-state representations before feature fusion is a promising direction for future research.

Overall, these experiments suggest that incorporating optimized trajectory information can improve trajectory tracking, while a dedicated trajectory-guided architecture offers further potential for balancing racing performance and interaction behavior.